

HIVE: A High-Priority Victim Cache for Accelerating GPU Memory Accesses

Yuhan Tang, Jianmin Zhang*, Sheng Ma*, Tiejun Li*, Hanqing Li, Shengbai Luo, Jixuan Tang, Lizhou Wu
National University of Defense Technology, Changsha, China
{jmzhang, masheng, tjli}@nudt.edu.cn

Abstract—The victim cache was originally designed as a secondary cache to handle misses in the L1 data (L1D) cache in CPUs. However, this design is often sub-optimal for GPUs. Accessing the high-latency L1D cache and its victim cache can lead to significant latency overhead, severely degrading the performance of certain applications. We introduce *HIVE*, a high-priority victim cache designed to accelerate GPU memory accesses. *HIVE* handles memory requests first, before they reach the L1D cache. Our experimental results show that *HIVE* achieves an average performance improvement of 77.1% and 21.7% compared to the baseline and the state-of-the-art architecture, respectively.

I. INTRODUCTION

Due to their powerful computing capacity and high energy efficiency, Graphics Processing Units (GPUs) are widely used in various applications, such as high-performance computing, machine learning and embedded systems. GPUs accelerate these applications by concurrently executing many warps, each consisting of a group of threads that execute the same instructions. For example, NVIDIA GPUs have supported up to 64 concurrent warps (2048 concurrent threads) within a single core since the Kepler architecture [1].

Unfortunately, a large number of warps can lead to severe performance degradation when GPUs execute memory-intensive applications [2]–[5]. One important reason for the performance degradation in these applications is that multiple warps sharing limited cache resources can cause severe cache contention [6], [7]. Although recent GPU architectures have larger L1D caches than previous generations, a single thread typically can access only tens of bytes in the L1D cache [8], [9].

Previous works have proposed using a victim cache to alleviate cache contention and achieve significant performance improvements in GPUs [7], [10]. In their designs, the function of the victim cache remains the same as in CPUs, acting as a secondary cache when a miss occurs in the L1D cache. However, we find that these designs are often sub-optimal for GPUs. This is due to two main reasons.

On the one hand, the L1D cache in GPUs exhibits higher miss rates, while evicted lines from the L1D cache have a high probability of being reused. This means that the number of memory requests that hit in the victim cache can approach or even exceed the number of requests that hit in the L1D cache. For example, we observe an average L1D cache miss rate of 63.7% in our evaluated GPU applications, and Oh et al. [7] also observe similar results. We also observe that most evaluated GPU applications exhibit a high probability (an average of 69.6%) of reusing evicted L1D cache lines (see Section II-B for more details). Therefore, the victim cache does not need to act as a supplement to the L1D cache.

On the other hand, GPUs use a unified architecture for the L1D cache and shared memory, or the L1D cache and texture cache, to save hardware overhead [1], [8], [9], [11]. However, this hardware architecture significantly increases the L1D hit latency. For example, the L1D hit latency is 32 cycles in the TU104 GPU and 82 cycles

in the GP100 GPU [12]. We observe that some applications are susceptible to the cache access latency, and increasing cache latency can severely degrade performance (see Section II-C for more details). However, the conventional victim cache is a secondary cache for the L1D cache. This means that the victim cache has an even higher hit latency, which severely degrades the performance of latency-sensitive applications.

To this end, we propose *HIVE*, a high-priority victim cache for accelerating GPU memory accesses. The key idea of *HIVE* is to prioritize using the victim cache rather than the L1D cache to handle memory requests, which means that the victim cache does not act as a supplement to the L1D cache. Doing so can improve performance because *HIVE* enables most memory requests to be fulfilled in the victim cache.

To implement *HIVE*, we insert an extra stage into the baseline pipeline, dedicated to handling memory accesses to the victim cache. This stage involves two hardware structures: the access controller and the victim cache tag array. The access controller is composed of set indexing logic and memory request queues: (1) the set indexing logic is responsible for calculating the victim cache set number based on the memory request address, and (2) the request queues buffer requests to access the victim cache, thereby avoiding the blocking of running warps from generating memory requests due to the limited number of requests that the tag array can handle simultaneously. The victim cache tag array is a traditional set-associative structure that stores tag information associated with the data array. To save hardware overhead, we use the method from the state-of-the-art architecture, Linebacker [7], reusing idle register files as the victim cache data array.

To improve the effectiveness of *HIVE*, we explore two optimization techniques on top of our basic design. First, we increase the effective capacity of the victim cache by employing a low-latency data compression technique. We find that a slight increase in hit latency in the high-priority victim cache due to data compression has almost no impact on performance. Second, We design a new replacement policy for the victim cache based on the LRU (Least Recently Used) algorithm, which minimizes the number of valid cache lines evicted.

To evaluate the effectiveness of *HIVE*, we measure the performance of various applications from different benchmark suites using the PTX mode in the AccelSim simulator [13]. Our experimental results show that *HIVE* achieves an average of 77.1% and 21.7% performance improvement compared to the baseline and the state-of-the-art architectures, respectively, with maximum improvements of 240.0% and 86.5%. As a trade-off, *HIVE* consumes 9.4% more power on average compared to the state-of-the-art architecture. However, *HIVE* can save an average of 17.4% in power consumption compared to the baseline architecture.

We make the following contributions:

- We propose *HIVE*, the first high-priority victim cache for accelerating GPU memory accesses. *HIVE* handles memory

*Corresponding authors.

requests first, before they reach the L1D cache. We have open-sourced our design code¹.

- We introduce two optimizations for HIVE: (1) increasing the effective capacity of the victim cache by using a low-latency data compression technique; (2) designing a cache replacement policy to minimize the number of valid cache lines evicted.
- We evaluate HIVE and show that it significantly improves performance and power efficiency, effectively outperforming the state-of-the-art across different GPU architectures.

II. BACKGROUND AND MOTIVATION

A. Baseline GPU Architecture

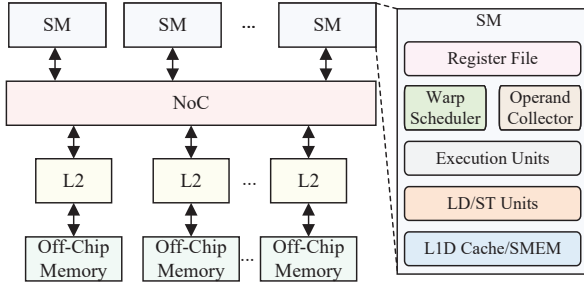


Fig. 1. Baseline GPU high-level organization.

GPU programming models allow the creation of thousands of threads that each execute the same code. NVIDIA’s programming model groups threads into 32-element vectors called warps to improve efficiency. The threads in each warp execute in a SIMT (single instruction, multiple thread) fashion. In most programs, many warps map to a single GPU core, or streaming multiprocessor (SM) in NVIDIA’s terminology [14].

Fig. 1 illustrates the high-level architecture of a baseline GPU. A GPU is composed of multiple SMs, in terms of the memory hierarchy, each of which contains a private register file, and a private, dynamically partitioned L1D cache and shared memory.

GPUs have very large register files to support a larger number of threads. Each active warp stores its architectural context in its own dedicated register file space. For example, NVIDIA GPUs have 256 kilobytes (KB) of register files per core across multiple generations [1], [8], [9], [11], [15]. However, this large register file is often underutilized [5], [7].

The L1D cache and shared memory (SMEM) share a unified on-chip memory space. The programmer specifies the capacity of software-managed shared memory, and the rest of the unified on-chip memory space is dedicated to the L1D cache, automatically [16]. Although recent GPU architectures have larger L1D caches than previous generations, a single thread typically can access only tens of bytes in the L1D cache [8], [9].

The GPU core accesses the memory-side shared L2 cache via the NoC in the case of an L1D cache miss. The L2 cache connects to off-chip memory, which is either HBM or GDDR [4]. Memory requests are expected to hit in the L1D cache to avoid the much longer latency of the L2 cache or off-chip memory accesses.

B. Capturing Lost Locality with Victim Cache

GPUs frequently evict cache lines that will be reused due to severe cache contention. To quantify the benefits of using a victim cache, we measure the L1D miss rates and the probability of evicted cache

lines from the L1D cache being reused in various applications (details of simulations and applications can be found in Section IV).

Table I. The L1D miss rates (*Miss.*) and the probability (*Prob.*) of evicted cache lines being reused.

App.	Miss.(%)	Prob.(%)	App.	Miss.(%)	Prob.(%)
KM	99.7	75.3	FW	48.8	72.2
GM	6.3	54.1	MV	91.7	70.6
GMV	79.1	85.2	BC	8.6	77.6
BG	79.2	67.8	SN	63.2	69.4
MVT	79.1	64.4	BFS	80.9	59.3

Table I shows our results. We make two key observations. First, all evaluated applications exhibit a high probability (54.1% - 85.2%) of reusing evicted cache lines, indicating that a victim cache can be effectively used to capture lost data locality. Second, the L1D cache of most applications exhibits high miss rates (48.8% - 99.7%), except for GM and BC, indicating that most memory requests cannot be fulfilled in the L1D cache. We conclude that if a victim cache is used, the number of memory requests that hit in the victim cache can approach or even exceed the number of requests that hit in the L1D cache for most applications.

C. Motivation for a High-Priority Victim Cache

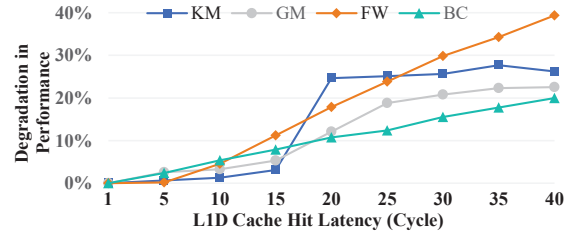


Fig. 2. Sensitivity of latency-sensitive applications to the L1D cache hit latency.

The increase in cache hit latency can significantly hurt the performance of some applications. We find that four out of ten evaluated applications experience a performance drop of over 20% when the L1D hit latency increases from 1 cycle to 40 cycles. We define these applications as latency-sensitive applications. Fig. 2 shows the sensitivity of latency-sensitive applications to the L1D cache hit latency. When the L1D cache latency increases to 40 cycles, the average performance drop of latency-sensitive applications is 27.0%.

The conventional victim cache was a secondary cache when a miss occurs in the L1D cache, which means that the hit latency of the victim cache includes the hit latency of the L1D cache. Even if memory requests hit in the victim cache, this latency overhead can severely degrade the performance of latency-sensitive applications.

Based on the above observations, it is natural to prioritize using a victim cache that does not act as a secondary cache to the L1D cache to handle memory requests. We refer to this victim cache as a high-priority victim cache. By doing so, most memory requests can be fulfilled in the victim cache, thereby accelerating overall memory accesses in GPUs.

III. DESIGN

In this section, we introduce HIVE, a high-priority victim cache designed to accelerate GPU memory accesses. HIVE handles memory requests first, before they reach the L1D cache.

¹<https://github.com/paper-account1/HIVE>

Fig. 4 shows the entry layout in the VTA. Each entry contains two sub-entries because we allow each warp register to store at most two compressed cache lines. Each sub-entry consists of fields including the current request’s tag, compression info (indicating how to decompress the compressed cache line), a valid bit (indicating if

the compressed cache line is valid), and LRU bits. Additionally, each entry has an active bit, which is set if the corresponding warp register is idle (determined at compile time).

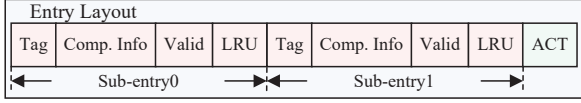


Fig. 4. VTA entry layout (*Comp. Info*: Compression Information, *ACT*: Active Bit).

We model the VTA using CACTI 6.5 [20] with a 22nm process. We observe that for a 256-way set-associative tag array with 4 sets, 1.877 ns (3 cycles) access latency and 0.041 nJ access energy are necessary for the configurations listed in Table III.

Replacement Policy. To reduce the number of valid compressed lines evicted from the victim cache, the VTA exploits a new cache replacement policy for the victim cache, as shown in Algorithm 1.

In the VTA, two sub-entries in a single entry map one-to-one to two compressed lines in a warp register. A compressed line is valid if the corresponding sub-entry in the VTA has its valid bit set. For any evicted line from the L1D cache to the victim cache, Algorithm 1 first avoids replacing an invalid sub-entry which could lead to the eviction of adjacent valid sub-entry.

For example, assume that a VTA entry stores an invalid sub-entry A and a valid sub-entry B. The sub-entry B is mapped to a compressed line of size 72 bytes. If a compressed cache line is evicted from the L1D cache and its size is 72 bytes, replacing the invalid sub-entry A will cause the valid sub-entry B to be evicted because the combined size of the two compressed cache lines will exceed 128 bytes. Algorithm 1 avoids this replacement.

Subsequently, Algorithm 1 attempts to find an entry with two invalid sub-entries and replaces one of them to store the information of the evicted compressed cache line. If no such entry is found, it finally uses the baseline LRU replacement policy.

Access Controller. The access controller contains two functional modules: the set indexing logic and the memory request queues. A memory request first arrives at the set indexing logic. To guarantee cache coherence, the VTA and the L1D cache use the same set indexing logic. The set indexing logic calculates the set number of the VTA for the memory request and forwards the request to the corresponding memory request queue.

The memory request queues buffer any request to the VTA, thereby avoiding the blocking of running warps from generating memory requests due to the limited number of requests that the tag array can handle simultaneously. A given request is dequeued as soon as the request's VTA set is ready to serve a new request. The number of memory request queues is equal to the number of sets in the VTA, as each set has a separate memory request queue. Similar to the L1D cache, the VTA can handle multiple requests per cycle from different memory request queues simultaneously.

Buffers. The write buffer for the register file (Write Buffer) (③) holds compressed data waiting to be written into the register file, and the decompression buffer (Decomp. Buffer) (⑦) holds the compression information needed for decompressing the compressed data. We evaluate the lengths of the write buffer and the decompression buffer from 1 to 20. We observe that a 1-entry write buffer and a 4-entry decompression buffer are sufficient to saturate performance.

Cache Coherence. The victim cache stores only the data evicted from the L1D cache, and the data evicted from the victim cache is directly discarded. When a store instruction is executed, HIVE

Algorithm 1: VTA Replacement Policy

Input : *tag* denotes the tag of evicted cache line
set denotes the set number of evicted cache line
sz denotes the compressed size of evicted cache line

Output: *pos* denotes the position of the sub-entry to be replaced
way denotes the way of the sub-entry to be replaced

// *i*₀ denotes sub-entry0, and *i*₁ denotes sub-entry1

```

1 for i ∈ VTA[set] do
2   if Valid(i0) = false then
3     if Valid(i1) and (sz + Size(i1) > 128B) then
4       continue
5     else
6       return pos ← Pos(i0), way ← Way(i)
7   else if Valid(i1) = false then
8     if Valid(i0) and (sz + Size(i0) > 128B) then
9       continue
10    else
11      return pos ← Pos(i1), way ← Way(i)
12 if find_invalid(tag, set, sz) then
13   pos, way ← invalid_replace(tag, set, sz)
14 else
15   pos, way ← lru_replace(tag, set, sz)
16 return pos, way

```

searches the requested line in the victim cache. If the written line hits in the victim cache, the victim cache entry is invalidated, and the store data is sent directly to the L1D cache. The invalidated entry can be reused as a victim line later. When a cache line is evicted from the L1D cache, the evicted line is sent to both the victim cache and the next level of the memory hierarchy. The result of this cache policy is that the data in the victim cache is never dirty, and the L1D cache and the victim cache do not store data from the same memory address simultaneously.

C. HIVE Workflow

An evicted cache line is sent to the BDI Compressor (①) for data compression. The compressor generates compression information during the compression, which is used to guide the decompressor to decompress the compressed data correctly. After compression, tag (*Tag*), set number (*SN*), compressed cache line (*Comp. Data*), and compression information (*Comp. Info*) are sent to the VTA (②). The VTA stores tag and compression information for tag probing and decompression, respectively. Then, the VTA calculates the register number associated with the compressed cache line. The register number (*RN*) and the compressed cache line are sent to the Write Buffer (③). Each buffer entry consists of a register number field and a compressed data field. When the port of the register file is free, the compressed cache line is written into the corresponding 128-byte warp register in the VDA (④). The VDA is a set of statically unused warp registers whose numbers are determined after compile time.

A memory request from the core first arrives at the Access Controller (⑤). The controller forwards the memory request to access the VTA (②). Upon a tag miss, the request is sent to the L1D cache of the MEM stage. Upon a tag hit, if the request is a store, the store data is sent to the L1D cache, and the corresponding sub-entry in the VTA is cleared. Upon a tag hit, if the memory request is a load, the register number is sent to the Arbitrator (⑥) for reading the compressed data, and the register number and the compression information are sent to the Decomp. Buffer (⑦) for decompressing the compressed data. Each buffer entry consists of a register number field and a compression information field. When the compressed data from

the VDA is sent to the BDI Decompressor (8), the decompression buffer forwards compression information to the BDI decompressor to indicate how to decompress the data correctly.

IV. EXPERIMENTAL SETUP

We model and evaluate HIVE using AccelSim [13], a cycle-level simulator. AccelSim has two execution modes: PTX and SASS. To obtain the necessary cache data, we use the PTX mode in the AccelSim simulator. Table III shows the simulation parameters modeling our baseline. This Turing-like GPU configuration is similar to that used in a recent study [4]. We select 10 applications from six benchmark suites: Rodinia-3.1 [21], ISPASS [22], Polybench [23], Tango [24], Microbenchmark [13], and Pannotia [25]. The applications used in our study are shown in Table IV.

Table III. Baseline GPU configuration.

Parameter	Value
GPU Core Features	30 SMT cores, 1.525GHz
Max Warps/SM	64 (2048 threads)
Warp Scheduling	4 GTO schedulers/SM
Register Files/SM	256KB, 16 banks
L1 Data Cache/SM	32/64KB 64/128-way L1 data cache, 4 banks, LRU, 128B cache line, access latency=38 cycles
L2 Cache	96KB/memory sub partition (6MB in total), access latency=198 cycles
Off-chip DRAM	RRD=3, RCD=12, RAS=28, RP=12, RC=40, CL=12, WR=10, access latency=298 cycles

Table IV. Evaluated applications.

App.	Abbr.	App.	Abbr.
kmeans [21]	KM	fw_block [25]	FW
gemm [23]	GM	spmv [25]	MV
gesummv [23]	GMV	bc [25]	BC
bigc [23]	BG	sn [24]	SN
mvt [23]	MVT	bfs [22]	BFS

We study GPU performance for four different architectures: (1) *BL*: the baseline system that models a GPU architecture with the parameters reported in Table III. (2) *LB*: the state-of-the-art architecture, Linebacker [7]. It dynamically throttles warps and utilizes idle register space as victim cache space. (3) *HIVE-Basic*: HIVE without BDI compressor and decompressor and using the baseline LRU replacement policy. (4) *HIVE-All*: the full design of HIVE. To provide a fair comparison, we require that HIVE and LB use the same maximum number of idle registers. To analyze the power consumption of HIVE, we use a modified version of AccelWattch [26] that is augmented with the VTA and BDI compressor/decompressor power models.

V. EVALUATION

A. Performance

To study the effectiveness of HIVE in improving GPU performance, we measure instructions per cycle (IPC) on four architectures, namely *BL*, *LB*, *HIVE-Basic*, and *HIVE-All* (see Section IV for details of each evaluated architecture). Fig. 5 shows the results. The y-axis shows the IPC of evaluated applications (the higher the better). All results are normalized to the baseline (*BL*).

We make four key observations. First, *HIVE-All* improves GPU performance by an average of 77.1% and 21.7% over *BL* and *LB*, respectively, for all applications. Remarkably, *HIVE-All* shows the greatest performance improvement in GMV (240.0% and 86.5%)

because GMV has sufficient idle registers that can be reused as victim cache space, and the compression algorithm effectively compresses the data. Additionally, although GM and BC exhibit lower L1D miss rates, they are latency-sensitive (see Section II-B and II-C), so HIVE still improves their performance compared to *BL* and *LB*.

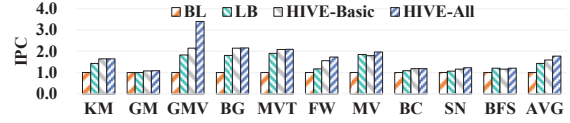


Fig. 5. Comparison of four GPU architectures' IPC (normalized to *BL*).

Second, *HIVE-Basic* improves performance by an average of 59.6% over *BL* by providing an extra on-chip cache and reducing access latency for most memory requests. Third, *HIVE-Basic* improves performance by an average of 11.2% over *LB* because it provides faster access to the victim cache. Fourth, *HIVE-All* improves performance by an average of 9.1% over *HIVE-Basic*, by utilizing data compression and exploiting a new replacement policy for the victim cache. We conclude that *HIVE-All* is highly effective at improving the performance of different applications.

B. Victim Cache Hit Rate

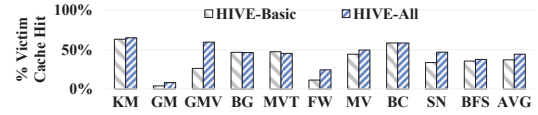


Fig. 6. Victim cache hit rates in *HIVE-Basic* and *HIVE-All*.

HIVE provides an extra cache to extend on-chip cache capacity, further reducing cache contention. We measure the victim cache hit rates for *HIVE-Basic* and *HIVE-All*. Fig. 6 shows our experimental results. We observe that the average hit rates for *HIVE-Basic* and *HIVE-All* are 37.1% and 44.2%, respectively, indicating that nearly half of the data evicted from the L1D cache can be reused in the victim cache. We also observe that *HIVE-All* improves hit rates by an average of 19.1% over *HIVE-Basic*, thanks to the BDI compression and the VTA replacement policy. We conclude that HIVE can effectively capture the lost data locality.

C. L2 Cache Traffic and Off-Chip Memory Traffic

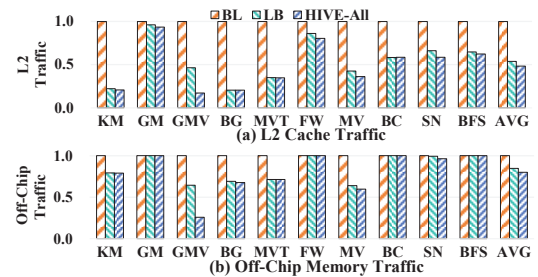


Fig. 7. L2 cache traffic and off-chip memory traffic in *BL*, *LB*, and *HIVE-All* (normalized to *BL*).

HIVE provides an extra victim cache and uses the BDI compression algorithm and the VTA replacement policy to increase the effective capacity of the victim cache, thereby effectively reducing access to the high-latency L2 cache and off-chip memory. We

compare the L2 cache traffic and the off-chip memory traffic of *HIVE-All* to those of *LB*. Fig. 7 shows our experimental results (normalized to the baseline (*BL*) architecture). *HIVE-All* reduces the L2 cache traffic and off-chip memory traffic by 51.7% and 20.0%, respectively, compared to the baseline. Compared to *LB*, *HIVE-All* reduces 12.0% more L2 cache traffic and 30.8% more off-chip memory traffic.

D. Overhead Analysis

We analyze the storage and power overheads of the additional hardware required by HIVE.

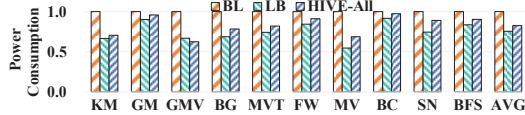


Fig. 8. Power consumption in *BL*, *LB* and *HIVE-All* (normalized to *BL*).

Storage Cost. HIVE has two main storage components: the VTA and the write buffer for the register file. The VTA is a 256-way set-associative tag array with 4 sets. Each entry in the VTA contains two sub-entries and one active bit. Each sub-entry contains 23-bit tag, 10-bit compression info, 1-bit valid, and 8-bit LRU. Consequently, the total size of the VTA is 10.625 KB. The write buffer has a 0.125 KB storage for compressed cache data. Overall, HIVE requires 10.75 KB of storage overhead, which accounts for 3.1% of the SM's storage.

Power Consumption. We measure the power consumption of *BL*, *LB*, and *HIVE-All*. Fig. 8 shows our experimental results (normalized to the baseline (*BL*) architecture). We observe that *HIVE-All* increases power consumption by an average of 9.4% compared to *LB*. However, *HIVE-All* significantly reduces power consumption by an average of 17.4% compared to the baseline. Both *HIVE-All* and *LB* reduce the L2 cache and the off-chip memory traffic compared to the baseline, but *HIVE-All* also increases register accesses and BDI compressor/decompressor accesses compared to *LB*.

Considering the experimental results in this section and the hardware cost estimation, we believe that HIVE is a cost-effective solution to improve performance on GPUs.

E. Sensitivity Study

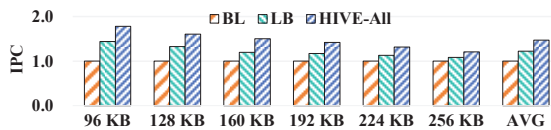


Fig. 9. Performance of varying unified memory space for the L1D cache and shared memory in *BL*, *LB*, and *HIVE-All* (all results are geometric mean values of the evaluated applications and normalized to *BL*).

Different GPU architectures have varying sizes of unified on-chip memory space for the L1D cache and shared memory [8], [15]. We explore the performance sensitivity by varying the unified on-chip memory space. We fixed the configurable shared memory capacity at 32/64 KB, with the exact size determined by the compiler. For example, if a kernel is configured with 32 KB of shared memory by the compiler in one architecture, the shared memory capacity for that kernel will still be configured as 32 KB by the compiler in other architectures. We measure the performance changes with the unified on-chip memory space for the L1D cache and shared memory set to 96 KB, 128 KB, 160 KB, 192 KB, 220 KB, and 256 KB.

Fig. 9 shows our results (normalized to the baseline (*BL*) architecture). We observe that *HIVE-All* improves performance by an average of 47.0% and 20.2% compared to the baseline and *LB*, respectively, across different unified memory space configurations. With the smallest capacity configuration (96 KB), *HIVE-All* shows 77.1% performance improvement while *LB* shows 43.6%. With the largest capacity configuration (256 KB), *HIVE-All* still outperforms *LB*. *HIVE-All* shows 20.8% performance improvement while *LB* shows 8.3%. We conclude that HIVE can be an efficient solution to improve cache efficiency across different GPU architectures.

VI. RELATED WORK

The victim cache, proposed by [27], is placed in the refill path of an L1D cache and stores the cache lines evicted from the L1D cache in CPUs. Subsequently, there have been various studies based on the victim cache [7], [28], [29]. Differently, our proposed HIVE is more suitable for GPUs because it reduces the hit latency for most memory requests.

Data compression has already been explored in GPUs. [18], [30]–[32] explore data compression in register files [30], interconnect networks [31], and caches [18], [32]. To better utilize the latency margin provided by the high-priority victim cache, HIVE adopts a low-latency BDI compression algorithm, whose effectiveness in GPUs has been validated in [18].

The register file in GPUs is fairly large but often underutilized. To address this problem, prior works propose new techniques to utilize the register file efficiently. [2] proposes a new technique that reuses the core's on-chip memories, including register files, to extend the LLC capacity for a running memory-bound workload. [33] proposes a new compiler technique that explores TLP and registers allocation trade-off with the compile-time workload characteristic analysis.

Warp throttling techniques are introduced to reduce cache contention in GPUs [7], [34]–[36]. They require empirically setting some thresholds to decide whether to throttle. This means that for some applications, warp throttling can even hurt the performance. As explained in earlier sections, the closest work to HIVE is Linebacker [7], which simply reuses idle registers as a victim cache and employs warp throttling technology. However, Linebacker's victim cache is inherited from CPUs and is suboptimal for GPUs. In contrast, HIVE is suitable for GPUs. It reduces the hit latency for most memory requests and increases the effective capacity of the victim cache. Therefore, HIVE is expected to be more feasible and applicable to real practices.

VII. CONCLUSION

We propose HIVE, a high-priority victim cache for accelerating GPU memory accesses. The key idea of HIVE is to prioritize using the victim cache rather than the L1D cache to handle memory requests, meaning the victim cache does not act as a supplement to the L1D cache. Our experimental results show that HIVE can significantly improve GPU performance across various applications and different architectures.

VIII. ACKNOWLEDGMENT

This work is supported in part by the National Key Research and Development Program of China under Grant 2022YFB2803405, in part by the National Natural Science Foundation of China under Grant 62472435, 62172430, and 62304257, in part by the STIP of Hunan Province under Grant 2022RC3065, in part by the Foundation of PDL under Grant 2023-JKWPDL-02, and in part by the Key Laboratory of Advanced Microprocessor Chips and Systems.

REFERENCES

- [1] "NVIDIA Kepler Architecture Whitepaper," <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-product-literature/NVIDIA-Kepler-GK110-GK210-Architecture-Whitepaper.pdf>.
- [2] S. Darabi, M. Sadrosadati, N. Akbarzadeh, J. Lindegger, M. Hosseini, J. Park, J. Gómez-Luna, O. Mutlu, and H. Sarbazi-Azad, "Morpheus: Extending the last level cache capacity in gpu systems using idle gpu core resources," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022.
- [3] A. Arunkumar, E. Bolotin, B. Cho, U. Milic, E. Ebrahimi, O. Villa, A. Jaleel, C.-J. Wu, and D. Nellans, "MCM-GPU: Multi-chip-module gpus for continued performance scalability," *ACM SIGARCH Computer Architecture News*, 2017.
- [4] X. Xu, L. Wang, L. Xiao, L. Liu, Y. Lv, X. Xie, M. Han, and H. Liu, "ATA-Cache: Contention mitigation for gpu shared l1 cache with aggregated tag array," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.
- [5] N. Jing, J. Wang, F. Fan, W. Yu, L. Jiang, C. Li, and X. Liang, "Cache-emulated register file: An integrated on-chip memory architecture for high performance gpgpus," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016.
- [6] G. Koo, Y. Oh, W. W. Ro, and M. Annavaram, "Access pattern-aware cache management for improving data utilization in gpu," in *Proceedings of the 44th annual international symposium on computer architecture*, 2017.
- [7] Y. Oh, G. Koo, M. Annavaram, and W. W. Ro, "Linebacker: Preserving victim cache lines in idle register files of gpus," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019.
- [8] "NVIDIA Ampere Architecture Whitepaper," <https://images.nvidia.cn/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>.
- [9] "NVIDIA H100 Tensor Core GPU Architecture Overview," <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper>.
- [10] E. M. Taylor and D. W. Chang, "Studying victim caches in gpus," in *2018 26th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP)*, 2018.
- [11] "NVIDIA Pascal Architecture Whitepaper," <https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf>.
- [12] Z. Jia, M. Maggioni, J. Smith, and D. P. Scarpazza, "Dissecting the nvidia turing t4 gpu via microbenchmarking," *arXiv preprint arXiv:1903.07486*, 2019.
- [13] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-sim: An extensible simulation framework for validated gpu modeling," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.
- [14] S. Damani, M. Stephenson, R. Rangan, D. Johnson, R. Kulkarni, and S. W. Keckler, "Gpu subwarp interleaving," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022.
- [15] "NVIDIA Turing Architecture Whitepaper," <https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf>.
- [16] S. Darabi, E. Yousefzadeh-Asl-Miandoab, N. Akbarzadeh, H. Falahati, P. Lotfi-Kamran, M. Sadrosadati, and H. Sarbazi-Azad, "OSM: Off-chip shared memory for gpus," *IEEE Transactions on Parallel and Distributed Systems*, 2022.
- [17] G. Pekhimenko, V. Seshadri, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, "Base-delta-immediate compression: Practical data compression for on-chip caches," in *Proceedings of the 21st international conference on Parallel architectures and compilation techniques*, 2012.
- [18] A. Arunkumar, S.-Y. Lee, V. Soundararajan, and C.-J. Wu, "LATTE-CC: Latency tolerance aware adaptive cache compression management for energy efficient gpus," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2018.
- [19] A. Arelakis, F. Dahlgren, and P. Stenstrom, "Hycomp: A hybrid cache compression method for selection of data-type-specific compression methods," in *Proceedings of the 48th International Symposium on Microarchitecture*, 2015.
- [20] N. Muralimanohar, R. Balasubramanian, and N. Jouppi, "Optimizing nuca organizations and wiring alternatives for large caches with cacti 6.0," in *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*, 2007.
- [21] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *2009 IEEE international symposium on workload characterization (IISWC)*, 2009.
- [22] A. Bakhoda, G. L. Yuan, W. W. Fung, H. Wong, and T. M. Aamodt, "Analyzing cuda workloads using a detailed gpu simulator," in *2009 IEEE international symposium on performance analysis of systems and software*, 2009.
- [23] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos, "Auto-tuning a high-level language targeted to gpu codes," in *2012 innovative parallel computing (InPar)*, 2012.
- [24] A. Karki, C. P. Keshava, S. M. Shivakumar, J. Skow, G. M. Hegde, and H. Jeon, "Tango: A deep neural network benchmark suite for various accelerators," in *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019.
- [25] S. Che, B. M. Beckmann, S. K. Reinhardt, and K. Skadron, "Pannotia: Understanding irregular gpgpu graph applications," in *2013 IEEE International Symposium on Workload Characterization (IISWC)*, 2013.
- [26] V. Kandiah, S. Peverelle, M. Khairy, J. Pan, A. Manjunath, T. G. Rogers, T. M. Aamodt, and N. Hardavellas, "Accelwatch: A power modeling framework for modern gpus," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021.
- [27] N. P. Jouppi, "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," *ACM SIGARCH Computer Architecture News*, 1990.
- [28] S. M. Khan, D. A. Jiménez, D. Burger, and B. Falsafi, "Using dead blocks as a virtual victim cache," in *Proceedings of the 19th international conference on Parallel architectures and compilation techniques*, 2010.
- [29] M. Zhang and K. Asanovic, "Victim replication: Maximizing capacity while hiding wire delay in tiled chip multiprocessors," in *32nd International Symposium on Computer Architecture (ISCA'05)*, 2005.
- [30] S. Lee, K. Kim, G. Koo, H. Jeon, W. W. Ro, and M. Annavaram, "Warped-compression: Enabling power efficient gpus through register compression," *ACM SIGARCH Computer Architecture News*, 2015.
- [31] N. Vijaykumar, G. Pekhimenko, A. Jog, A. Bhowmick, R. Ausavarungnirun, C. Das, M. Kandemir, T. C. Mowry, and O. Mutlu, "A case for core-assisted bottleneck acceleration in gpus: enabling flexible data compression with assist warps," *ACM SIGARCH Computer Architecture News*, 2015.
- [32] J. Kim, M. Sullivan, E. Choukse, and M. Erez, "Bit-plane compression: Transforming data for better compression in many-core architectures," *ACM SIGARCH Computer Architecture News*, 2016.
- [33] X. Xie, Y. Liang, X. Li, Y. Wu, G. Sun, T. Wang, and D. Fan, "Enabling coordinated register allocation and thread-level parallelism optimization for gpus," in *Proceedings of the 48th International Symposium on Microarchitecture*, 2015.
- [34] T. G. Rogers, M. O'Connor, and T. M. Aamodt, "Cache-conscious wavefront scheduling," in *2012 45th Annual IEEE/ACM International Symposium on Microarchitecture*, 2012.
- [35] S.-Y. Lee and C.-J. Wu, "CAWS: Criticality-aware warp scheduling for gpgpu workloads," in *Proceedings of the 23rd international conference on Parallel architectures and compilation*, 2014.
- [36] B. Wang, Y. Zhu, and W. Yu, "OAWS: Memory occlusion aware warp scheduling," in *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation*, 2016.